

# **PROJECT REPORT**

## **2048 Game**

**Name:** - Navya Shukla

**Registration no.:** - 25MIP10161

### **1. Introduction:**

2048 is a number combination puzzle based on a grid. The player moves tiles in four directions, and equal adjacent tiles combine into a tile containing their sum. After a successful move, a new tile is placed in an empty position. This project implements the core mechanics in Java while adding a formatted board, colored output, a title header, score display, controls, and styled win, game over, and exit messages.

### **2. Objectives:**

- Develop a functional 2048 puzzle game using Java.
- Implement a  $4 \times 4$  game board using a two-dimensional array.
- Allow movement in up, down, left, and right directions.
- Combine tiles with equal values.
- Maintain and update the player's score.
- Generate random tiles after successful moves.
- Detect when the 2048 tile has been reached.
- Detect when no further valid move is possible.
- Provide a visually enhanced console interface using ANSI colors.

### **3. Technologies Used:**

| Technology / Class    | Purpose              | Use in Project                                  |
|-----------------------|----------------------|-------------------------------------------------|
| Java                  | Programming language | Implements the complete game logic.             |
| Scanner               | User input           | Reads W, A, S, D, and Q commands.               |
| Random                | Randomization        | Selects empty cells and generates 2 or 4 tiles. |
| ArrayList             | Data collection      | Stores coordinates of empty board cells.        |
| ANSI Escape Sequences | Terminal styling     | Adds colors and formatted output.               |

### **4. System Requirements:**

- Java Development Kit (JDK).
- Java compatible terminal or IDE terminal.
- Code editor or Java IDE.

- Terminal supporting ANSI escape sequences for enhanced colors.

## 5. Game Working:

1. Initialize the  $4 \times 4$  board and score.
2. Generate two initial random tiles.
3. Clear the screen and display the header, board, score, and controls.
4. Read the player's W, A, S, D, or Q command.
5. Perform the corresponding movement function.
6. If the board changes, generate a new random tile.
7. Check whether 2048 has been reached.
8. Check whether another valid move is possible.
9. Continue until the player quits, wins, or reaches game over.

## 6. Main Functions:

| Function        | Functionality                                                              |
|-----------------|----------------------------------------------------------------------------|
| main()          | Controls initialization, input, movement selection, and game state checks. |
| printHeader()   | Displays the stylized 2048 title and Java Edition label.                   |
| printBoard()    | Displays the $4 \times 4$ board with borders and tile values.              |
| getTileColor()  | Returns a terminal color according to tile value.                          |
| addRandomTile() | Finds empty cells and inserts a randomly selected 2 or 4.                  |
| moveLeft()      | Compresses and merges tiles toward the left.                               |
| moveRight()     | Uses row reversal with left movement.                                      |
| moveUp()        | Uses transposition with left movement.                                     |
| moveDown()      | Uses transposition and reversal with left movement.                        |
| reverseRows()   | Reverses each board row.                                                   |
| transpose()     | Swaps rows and columns.                                                    |
| hasWon()        | Checks whether a cell contains 2048.                                       |
| canMove()       | Checks empty cells and adjacent equal tiles.                               |
| clearScreen()   | Clears the terminal display.                                               |
| pause()         | Waits for Enter after invalid input.                                       |

## 7. User Controls:

| Key | Action     |
|-----|------------|
| W   | Move Up    |
| S   | Move Down  |
| A   | Move Left  |
| D   | Move Right |
| Q   | Quit Game  |
|     |            |

## 8. Algorithm:

1. Initialize board and score.
2. Place two random tiles.
3. Display the board.
4. Read the player's command.
5. Exit if Q is entered.
6. Perform the selected movement.
7. Generate a new tile after a successful movement.
8. Check for 2048.
9. Check for possible moves.
10. Repeat until the game ends.

## 9. Future Enhancements:

- Develop a GUI using Java Swing or JavaFX.
- Add actual arrow key controls.
- Add a restart button.
- Store and display the highest score.
- Support different board sizes.
- Add animations and sound effects.
- Add save/load functionality.
- Add a leaderboard system.
- Add difficulty levels.

## 10. Learning Outcomes:

- Java programming fundamentals.
- Two-dimensional arrays and array manipulation.
- Loops and conditional statements.
- Methods and modular programming.
- Scanner based user input.
- Random number generation.
- ArrayList usage.
- Game state management.
- Basic algorithm design and problem solving.
- ANSI escape sequences for console presentation.

## 11. Conclusion:

The 2048 Game is a practical application of Java programming concepts in an interactive puzzle game. The project implements a  $4 \times 4$  board, random tile generation, four direction movement, tile merging, score calculation, win detection, and game-over detection. The enhanced terminal presentation improves the user experience through colors, formatted controls, a title header, and styled game messages. The project also provides a foundation for future development into a graphical Java game.